

CPS 200

Lecture 06

Jan 28, 2026

Kyu Hur

Schedule

Date	Session	Quiz	Topic
Jan 5			Reading Day
Jan 7	1	No	Intro to Programming I
Jan 12			National Holiday
Jan 14	2	No	Think Python: Programming as a way of thinking, Variables and Statements
Jan 19	3	Yes	Think Python: Functions, Functions and Interfaces
Jan 21	4	Yes	Think Python: Functions and Interfaces
Jan 26	5	Yes	Think Python: Conditionals and Recursion
Jan 28	6	Yes	Think Python: Return Values
Feb 2	7	Yes	Think Python: Iteration and Search
Feb 4	8	Yes	Think Python: Strings and Regular Expressions

From Lecture 05

Bonus Question 3

- Write code that creates a turtle, then uses a for loop to draw 36 squares at size 100, rotating the turtle 10 degrees left after each square. You do not have to define the `square()` function

```
jupyturtle.make_turtle()  
  
for i in range(36):  
    square(100)  
    left(10)
```

```
def square(num):  
    for i in range(4):  
        forward(num)  
        left(90)
```

Nested Conditionals

- Write a nested conditional statement. Assume x is a random integer, between $-\infty$ to ∞ . Work with your partners.
- Print a conditional message that states one of the three following statements based on the value of x :
 - “ x is equal to 0”
 - “ x is bigger than 0”
 - “ x is smaller than 0”

Modulus

- What is the answer to the following questions if $x = 10$? Solve with your partners.
 - $x \% 2$ 0
 - $x \% 10$ 0
 - $x \% 20$ 10
 - $x \% -3$ -2
 - $x \% -4$ -2

and / or keywords

- Solve the following questions with your partners if $x = 12$ and $y = 0$.
 - $x == 12$ and $y == 0$ True
 - $x == 12$ or $y == 0$ True
 - $x == 12$ and $y == 1$ False
 - $x == 12$ or $y == 1$ True
 - $x == 13$ or $y == 0$ True

and / or keywords

- Solve the following questions with your partners.
 - True and True True
 - True or True True
 - True or False True
 - False or False False
 - True and False False
 - False and False False

Recursion

- What is recursion? Please explain recursion to your partners.
- Recursion is a technique where a function calls itself, to solve a problem by breaking it down into smaller, self-similar subproblems
 - The condition that terminates the recursion is the **base case**
 - The part of the function that calls itself with a different input is the **recursive step**

Recursion

- Recursion is conceptually that same as **nested functions**

```
def f_1 () :  
    f_2 ()
```

```
def f_2 () :  
    f_3 ()
```

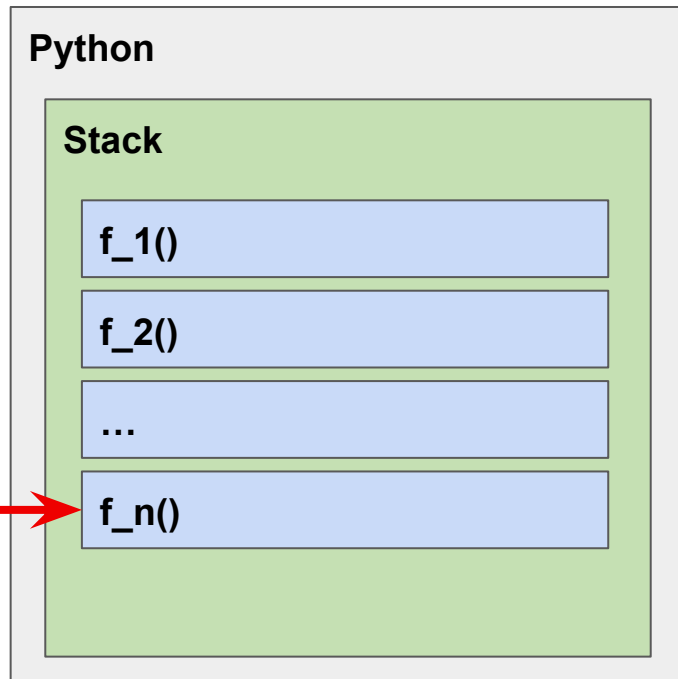
```
def f_3 () :  
    return None
```

```
f_1 ()
```

Recursion

- By default, Python allows 1000 **frames** in the stack
- Exceeding this will give you a `RecursionError`, commonly known as a **stack overflow**

If there are >1000 frames here, it will give you an error



Recursion

- Write a function called `countdown()` that takes an integer parameter `num`. It should print down from `num` to 1, then print "Blastoff!"
 - Can you write this with a for loop?
 - Can you write this as a recursive function?
- Pass the following to `countdown()` as the argument:
 - 100
 - 975
 - 976
 - 1000

Recursion

- Now, write a function similar to `countdown()` but it only counts down even numbers (`countdown_even_nums()`). How would you do this?
 - **Hint:** Remember the modulus operator (%)
- Could you write a function that counts down by multiples of 3? How about multiples of 3 OR 5?

Lecture 06

Return Values

- Until now, we relied on `print()` to see values
- However, `print()` is only for us to see / peek at intermediary values
- To assign outputs of functions to variables, we need the `return` **reserved keyword**

Return Values

- What is the output of the left vs the right? What is different? Discuss with partners.

```
def multiply_by_2(num):  
    return num * 2
```

```
val = 2  
val = multiply_by_2(val)  
print(val)
```

```
def multiply_by_2(num):  
    print(num * 2)
```

```
val = 2  
multiply_by_2(val)  
print(val)
```


Return Values

- When a function returns a value, we call that a **fruitful** function
- When a function does not return a value, we call that a **void** function
 - Some other languages (like C) require that we specify what we return; if we return nothing you have to specify it as **void**
 - We do not have to do this in Python but keep this in mind
- Returning is NOT the same as printing!

Return Values and Conditionals

- You can adjust what is returned from a function with **conditionals** (i.e. if, elif, else)
- What is the output of the following code? Why?

```
def function(num):  
    if num % 2 == 0:  
        return num * 2  
    else:  
        return num
```

```
val1 = function(2)  
val2 = function(3)  
print(val1, val2)
```

Think Python

Think Python: Return Values

- [Chapter 06.ipynb](#)

End